PROGRAMACIÓN BÁSICA
UNIDAD 1

Bienvenidos a la unidad 1: "Introducción a la Programación Orientada a Objetos". Los contenidos que te iremos presentando son netamente introductorios ya que pretenden darte los conceptos básicos que necesitarás conocer para poder leer y comprender los temas siguientes.

Por lo anterior, la complejidad con la que se tratan los temas aquí expuestos es de un nivel inicial. Cada uno de estos temas será abordado en detalle llegado el momento, en las unidades que así lo requieran.

Al finalizar el estudio de esta unidad habrás obtenido conocimientos teóricos acerca de:

- El paradigma orientado a objetos. Propiedades
- Concepto de clase, objeto.
- Características de un objeto. Atributos
- Comportamiento de un objeto. Métodos.
- Introducción a la programación orientada objetos en Java.

I. Introducción

La Programación Orientada a Objetos (POO) es un paradigma de programación basado en los conceptos de clase y objeto. Thomas Kuhn describe el concepto de paradigma como un conjunto de teorías y métodos que juntos representan un medio de organización del conocimiento.

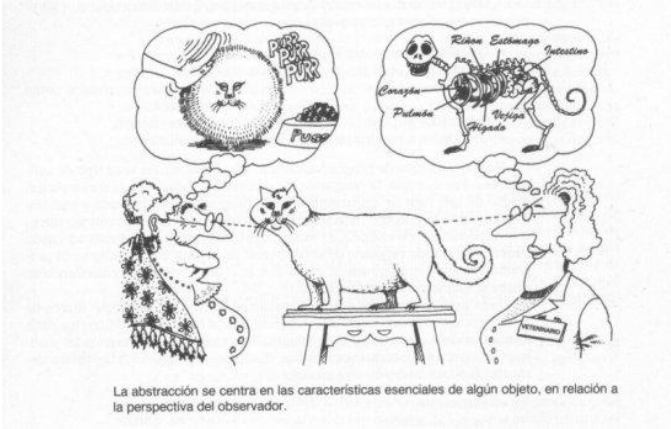
En POO, las entidades centrales son los objetos, que encapsulan con el mismo nombre estructuras de datos y operaciones o algoritmos que manipulan esos datos. En este sentido, un objeto es una entidad que cuenta con un conjunto de atributos y métodos.

La orientación a objetos puede describirse como el conjunto de disciplinas que desarrollan y modelizan software que facilitan la construcción de sistemas complejos a partir de componentes más pequeños. La orientación a objetos se basa en la modelización en términos de objetos. Su soporte fundamental es el *modelo objeto*.

1. PROPIEDADES de la orientación a objetos

❖ Abstracción

La abstracción es la propiedad que permite representar las características esenciales de un objeto, sin preocuparse de las restantes características (no esenciales). Una abstracción se centra en la vista externa de un objeto, de modo que sirva para separar el comportamiento esencial de un objeto de su implementación.

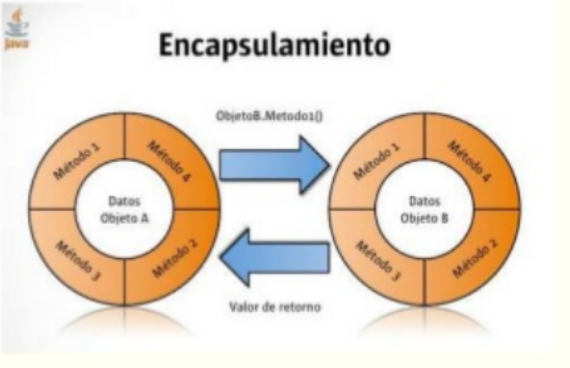


La abstracción se centra en las características esenciales de algún objeto, en relación a la perspectiva del observador.

El elemento clave de la POO es la clase. Una clase se puede definir como una descripción abstracta de un grupo de objetos, cada uno de los cuales se diferencia por su *estado* específico y por la posibilidad de realizar una serie de *operaciones.*

❖ Encapsulamiento

El encapsulamiento es la propiedad que permite asegurar que el contenido de la información de un objeto está oculta al mundo exterior. El encapsulamiento, es el proceso de ocultar todos los secretos de un objeto que no contribuyen a sus características esenciales.

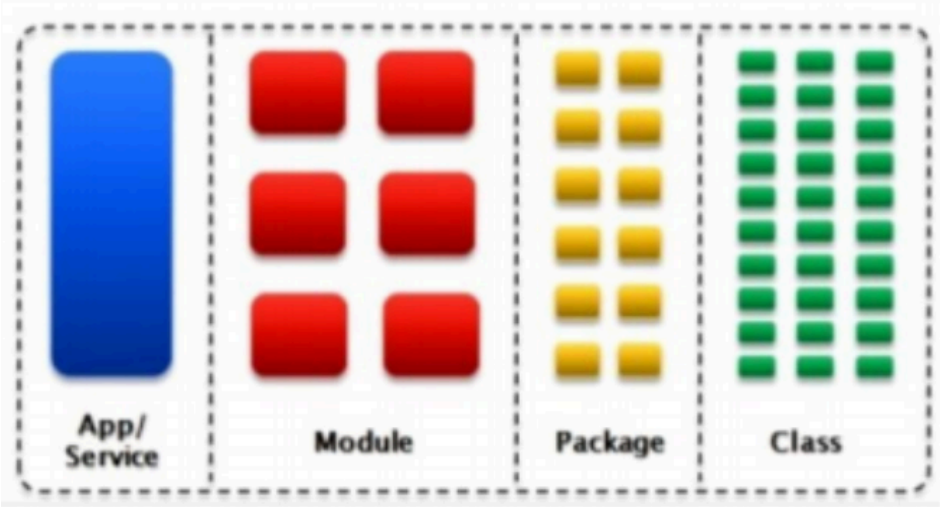


El encapsulamiento permite dividir un programa en módulos. Estos módulos se implementan mediante clases, de forma que una clase representa el encapsulamiento de una abstracción. En la práctica, esto significa que cada clase debe tener dos partes: una interfaz y una implementación. La *interfaz* de una clase captura solo su vista externa y la *implementación* contiene la representación de la abstracción, así como los mecanismos que realizan el comportamiento deseado.

❖ Modularidad

La modularidad es la propiedad que permite subdividir una aplicación en partes más pequeñas llamadas módulos, cada una de las cuales debe ser tan independiente como sea posible de la aplicación en sí y de las restantes partes. En algunos lenguajes de programación, los módulos se definen como *paquetes* (package) y/o con el uso de funciones.

En el lenguaje de programación Java y en algunos entornos de desarrollo integrado (IDE), la modularidad radica en la separación de todos los posibles paquetes de código. La siguiente imagen muestra la división que se hace a partir de una aplicación base o un servicio general, que se divide en diferentes módulos con un propósito especial y finalmente un subdivisión en pequeños paquetes de código que integran las clases.



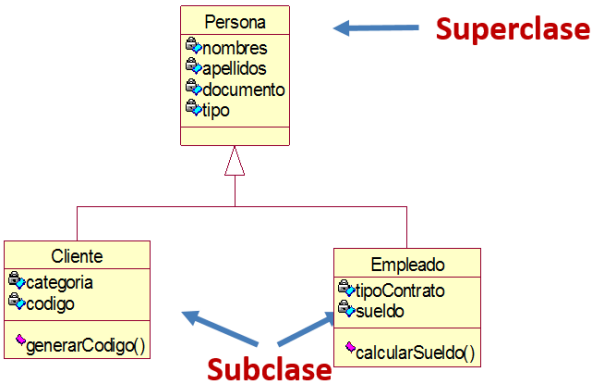
La modularidad es la propiedad de un sistema que permite su descomposición en un conjunto de módulos cohesivos y débilmente acoplados.

❖ Jerarquía

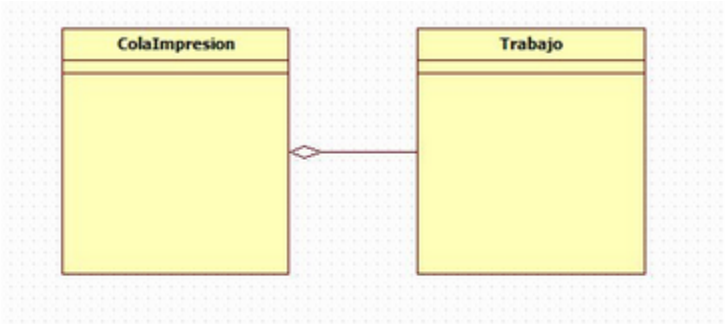
La *jerarquía* es una propiedad que permite una ordenación de las abstracciones.

Las dos jerarquías más importantes de un sistema complejo son:

- **Estructura de clases** (jerarquía *es un*), las cuales se representan mediante de relaciones de generalización / especialización o herencia;



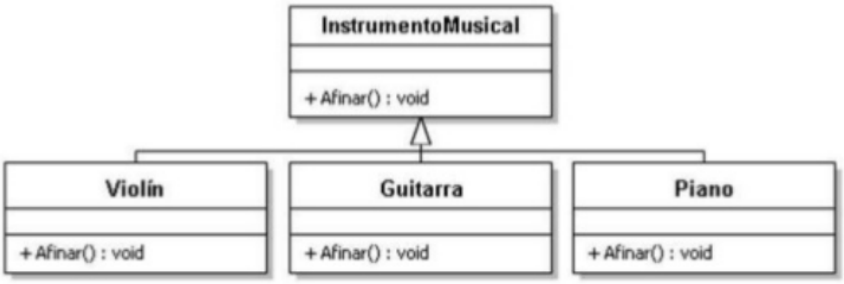
- **Estructura de objetos** (jerarquía *parte de*), la cual se define mediante relaciones de agregación.



Los trabajos forman parte de la cola de impresión

❖ Polimorfismo

El *polimorfismo* es la propiedad que permite la implementación especializada de comportamiento (métodos) a través de una jerarquía de clases. Por ejemplo, si tenemos la clase InstrumentoMusical con un método llamado Afinar(), cada clase hija puede implementar su propio Afinar() utilizando siempre este mismo método. De esta forma, se puede utilizar cualquiera de estas clases hijas a la hora de llamar a este método.



Son comportamientos diferentes asociados a objetos distintos. Pueden compartir el mismo nombre y, al llamarlos por ese nombre, se utilizará el comportamiento correspondiente al objeto que se esté usando.

2. CLASIFICACIÓN de los lenguajes orientados a objetos

Es posible clasificar los lenguajes de programación orientados a objetos de acuerdo con sus características. En este sentido, es posible identificar tres categorías:

1. Lenguajes *basados en objetos*: soportan objetos. Es decir, disponen de componentes caracterizados por un conjunto de operaciones y un estado.
2. Lenguajes *basados en clases*: que implican objetos y clases. Es decir, disponen de componentes tipo clase con operaciones y estado común.
3. Lenguajes *orientados a objetos:* que además de objetos y clases ofrecen mecanismos de herencia entre clases. Esto es, la posibilidad de derivar operaciones y atributos de una clase (superclase) a sus subclases.

3. PROGRAMACIÓN Orientada a Objetos (POO)

Según Grady Booch, la Programación Orientada a Objetos (POO) es un método de implementación en el que los programas se organizan como colecciones cooperativas de objetos, cada uno de los cuales representa una instancia de alguna clase, y cuyas clases son todas miembros de una jerarquía de clases unidas mediante relaciones de herencia.

❖ El Objeto

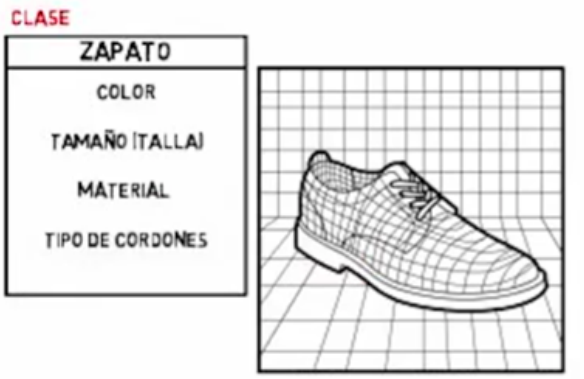
La idea fundamental en los lenguajes orientados a objetos es combinar en una sola unidad *datos y funciones que operan sobre esos datos*. Tal unidad se denomina *objeto*. Las funciones del objeto son el único medio para acceder a los datos privados de un objeto. De este modo, los datos están ocultos, lo que asegura que no se puedan modificar accidentalmente por funciones externas al objeto.

Los datos y las funciones están encapsulados en una única entidad o módulo. El *encapsulamiento* de datos y el *ocultamiento* de objetos son términos importantes en los lenguajes orientados a objetos. Si se desea modificar los datos de un objeto, se conoce exactamente cuáles son las funciones que interactúan con el mismo. Ninguna otra función puede acceder a los datos. Esta característica simplifica la escritura, depuración y mantenimiento del programa.

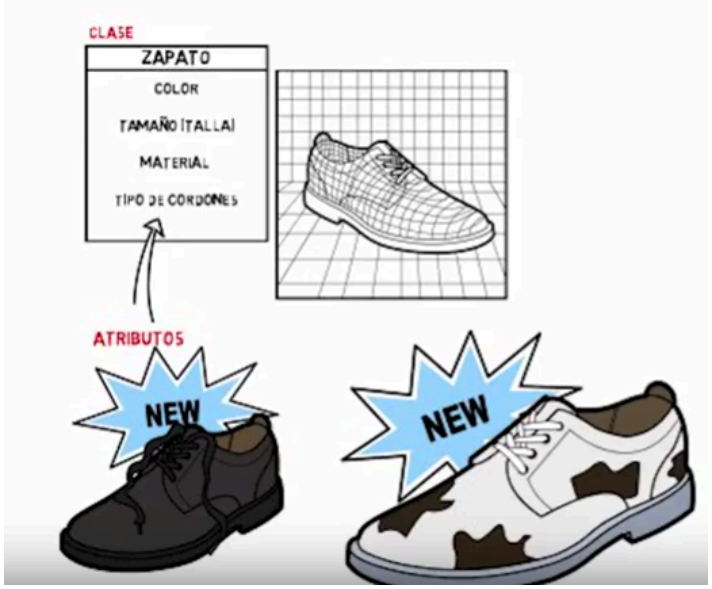
❖ La Clase

Una clase es la descripción de un conjunto de objetos; consta de métodos y datos que resumen características comunes de un conjunto de objetos. Se pueden definir muchos objetos de la misma clase. Dicho de otro modo, una clase es la declaración de un tipo de objeto.

Las clases son similares a los tipos de datos y equivalen a modelos o plantillas que describen cómo se construyen ciertos tipos de objetos.



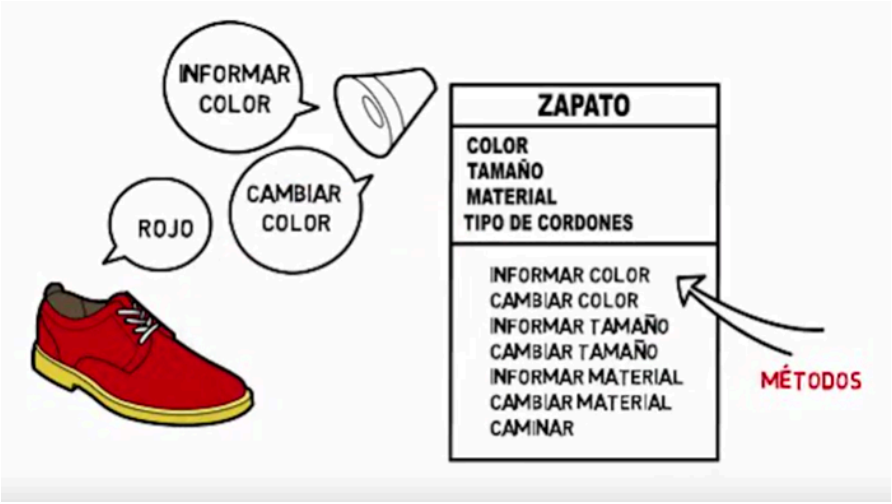
Cada vez que se construye un objeto a partir de una clase, estamos creando lo que se llama una *instancia* de esa clase. Por consiguiente, los objetos no son más que instancias de una clase. Una *instancia* es una variable de tipo objeto. En general, instancia de una clase y objeto son términos intercambiables.



Cada vez que se construye un objeto de una clase, se crea una instancia de esa clase. Los objetos se crean cuando un mensaje de petición de creación se recibe por la clase base.

❖ Métodos y mensajes

Un programa orientado a objetos consiste en un número de objetos que se comunican unos con otros llamando a funciones miembro. Las funciones miembros también se denominan *métodos.*



Los procedimientos y funciones, denominados *métodos* o *funciones miembro*, residen en el objeto y determinan cómo actúan los objetos cuando reciben un mensaje. Un *mensaje* puede verse como la

llamada a alguna acción que hace un objeto. Un método especifica cómo se ejecuta el mensaje. El conjunto de mensajes a los cuales puede responder un objeto se denomina *protocolo* del objeto.

Al igual que en las cajas negras, la estructura interna de un objeto está oculta a los usuarios y programadores. Los mensajes que recibe el objeto son los únicos conductos que conectan el objeto con el mundo externo. Los datos de un objeto están disponibles para ser manipulados sólo por los métodos del propio objeto.

Cuando se ejecuta un programa orientado a objetos, ocurren tres sucesos. Primero, los objetos se crean a medida que se necesitan. Segundo, los mensajes se mueven de un objeto a otro (o desde el usuario a un objeto) a medida que el programa procesa información internamente o responde a la entrada del usuario. Tercero, cuando los objetos ya no son necesarios, se borran y libera la memoria.

Definición de objetos

Un objeto es una abstracción de cosas (entidades) del mundo real, tales que:

- Todas las cosas del mundo real dentro de un conjunto, denominadas instancias, tienen las mismas características.
- Todas las instancias tienen las mismas reglas
- Cada objeto consta de:
    - Estado (atributos)
    - Operaciones o comportamiento (métodos invocados por mensajes).

Desde el punto de vista informático, los objetos son *tipos abstractos de datos* (tipos que encapsulan datos y funciones que operan sobre esos datos). Algunos ejemplos típicos de objetos:

- Número racional
    - Estado: valor actual
    - Operaciones: sumar, multiplicar, asignar
- Vehículo

  - Estado: velocidad, posición, precio
  - Operaciones: acelerar, frenar, parar
- Conjunto
  - Estado: elementos
  - Operaciones: añadir, quitar, visualizar
- Avión
  - Estado: fabricante, modelo, matrícula, número de pasajeros
  - Operaciones: aterrizar, despegar, navegar

Identificación de objetos

El primer problema que se nos plantea al analizar un problema que se desea resolver mediante un programa orientado a objetos es *identificar los objetos.*

La identificación de objetos se resuelve examinando la descripción del problema (análisis gramatical somero del enunciado o descripción) y localizando los nombres o cláusulas nominales. Normalmente, estos nombres y sus sinónimos se suelen escribir en una tabla de la que luego deduciremos los objetos reales. Los objetos, según Shlaer, Mellor y Coad/Yourdon, pueden caer dentro de las siguientes categorías:

- Cosas tangibles: avión, reactor nuclear, fuente de alimentación, televisor, libro, automóvil.
- Roles o papeles jugados o representados por personas: gerente, cliente, empleado, médico, paciente, ingeniero.
- Organizaciones: empresa, división, equipo
- Incidentes: representa un suceso, evento u ocurrencia; tales como vuelo, accidente, suceso, llamada a un servicio de asistencia técnica.
- Interacciones: implican generalmente una transacción o contrato y relacionan dos o más objetos del modelo. Por ejemplo: compras (comprador, vendedor, artículo).
- Especificaciones: muestran aplicaciones de inventario o fabricación (televisor, heladera)
- Lugares: sala de embarque, muelle de carga

Una vez identificados los objetos, será necesario indicar los atributos y las operaciones que actúan sobre ellos. Los *atributos* describen la abstracción de características individuales que poseen todos los objetos.

AVION
Matrícula Licencia de piloto Nombre de avión Capacidad de carga Número de pasajeros máximo

EMPLEADO
Nombre Número de identificación Salario Dirección Nombre del departamento

Las *operaciones* cambian al objeto, su estado, de alguna forma. Es decir, cambian valores de uno o más atributos contenidos en el objeto. Aunque existe un gran número de operaciones que se pueden realizar sobre un objeto, generalmente se dividen en tres grandes grupos:

- Operaciones que *manipulan* los datos de alguna forma específica (añadir, borrar, cambiar formato).
- Operaciones que realizan un *cálculo o proceso*
- Operaciones que comprueban (*monitorizan*) un objeto frente a la ocurrencia de algún suceso de control

La identificación de las operaciones se realiza haciendo un nuevo análisis gramatical de la descripción del problema y buscando y aislando los verbos del texto.

Composición de Objetos

Una de las características que hacen a los objetos ser muy potentes es que pueden contener otros objetos (como atributos). Los objetos que contienen otros objetos se conocen como *objetos compuestos*.

En la mayoría de los sistemas, los objetos compuestos no contienen en sentido estricto otros objetos, sino que contienen variables que se refieren a otros objetos. La referencia almacenada en la variable se llama identificador del objeto. Esta característica ofrece dos ventajas importantes:

1. Los objetos contenidos pueden cambiar en tamaño y composición, sin afectar al objeto compuesto que los contiene. Esto hace que el mantenimiento de sistemas complejos de objetos anidados sea más sencillo.
2. Los objetos contenidos están libres para participar en cualquier número de objetos compuestos, en lugar de estar bloqueado en un único objeto compuesto.

Un objeto compuesto consta de una colección de dos o más objetos componentes. Los objetos componentes tienen una relación *parte de* o *componente de* con el objeto compuesto. Cuando un objeto compuesto se instancia para producir una instancia del objeto, todos sus objetos componentes se deben instanciar al mismo tiempo. Cada objeto componente puede ser a su vez un objeto compuesto.

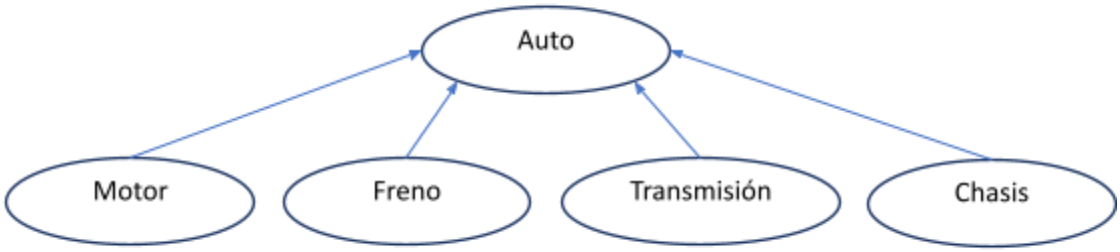


Figura 4 Relación de agregación (parte de)

La relación *parte de* puede representarse también como *tiene un,* que indica la relación que une al objeto *agregado* o *continente*. En el caso del objeto compuesto Auto, se leerá: *Auto tiene un Motor,* etc.

II. Programación Orientada a Objetos en Java

Java es un lenguaje de programación ampliamente aceptado en la actualidad, el cual implementa los conceptos de la POO para el desarrollo de distintos tipos de aplicaciones. Empleando Java es posible desarrollar aplicaciones Web, de escritorio, para dispositivos móviles, o para cualquier tipo de dispositivo para el que se haya compilado la Máquina Virtual de Java (JVM por sus siglas en inglés).

Desarrollaremos mediante un programa sencillo que implementa la jerarquía de clases que se muestra en la Figura 1, la sintaxis provista por el lenguaje para implementar los conceptos presentados en las secciones anteriores.

Figura 1. Jerarquía de clases de figuras geométricas

1. DECLARACIÓN de una Clase y sus Atributos

Tal como se indicó previamente, una clase es una representación de la estructura de datos y el comportamiento compartidos por un conjunto de objetos, los cuales se diferencian entre sí por su estado. En esta sección se describe cómo es posible definir clases y sus atributos en Java. La definición del comportamiento se detallará en la siguiente sección.

En el ejemplo de la Tabla 1, se muestra la declaración de una clase Rectángulo en Java. En línea 1, se indica el paquete en el cual se ha declarado la clase. Los paquetes (packages) son el mecanismo provisto por Java para organizar clases. Los paquetes permiten agrupar clases del mismo modo que los directorios y subdirectorios permiten agrupar y organizar archivos. Si bien la declaración de un paquete

no es obligatoria, esto consiste en una buena práctica especialmente cuando el número de clases definidas crece.

Las líneas 3 a 7 muestran un ejemplo de comentario multilínea en Java. Los comentarios permiten introducir notas aclaratorias respecto del propósito de una clase, atributo, función o cualquier otro fragmento de código. Los comentarios frecuentemente incluyen palabras clave precedidas por el carácter arroba, tales como @author en la línea 6. Estas palabras clave no tienen ningún efecto en la declaración de la clase, pero son empleados por los IDEs como Netbeans para mostrar información acerca de las clases y sus elementos a los usuarios.

En la línea 8 se puede ver un ejemplo de declaración de clase. Esto se hace mediante la palabra reservada *class*. En el ejemplo, la clase Rectángulo es declarada como pública (*public*). Esto significa que la clase es accesible desde cualquier paquete. En caso de no incluir la palabra *public* en la declaración de la clase, la misma será solamente accesible desde clases declaradas desde el mismo paquete. En Java, las clases se almacenan en archivos con extensión .java. Cada archivo puede contener solo una clase pública, y esta clase debe tener el mismo nombre que el archivo. Es decir, que la clase del ejemplo debe guardarse en un archivo llamado Rectangulo.java.

```
1.    package poo.figuras.geometricas;
2.
3.    /**
4.    Ejemplo de clase concreta Rectángulo
5.
6.    @author TSTI POO
7.    */
8.    public class Rectangulo {
9.       /*
10.      Atributo protegido (encapsulado) base
11.      */
12.      protected double base;
13.
14.      /**
15.      Atributo protegido (encapsulado) altura
16.      */
17.      protected double altura;
18.
19.      /**
20.      Constructor de la clase Rectangulo
21.      */
22.      public Rectangulo(){
23.         this.base = 2;
24.         this.altura = 1;
25.      }
26.   }
```

Tabla 1 Declaración de clase Rectángulo y sus atributos en Java

Las líneas 12 y 17 muestran ejemplos de declaración de atributos de clase en Java. Los atributos base y altura de la clase Rectángulo tienen nivel de acceso protegido (protected). Esto significa que dichos atributos solamente pueden ser accedidos desde la funciones en la clase Rectangulo y clases que hereden los atributos de la misma. Otros niveles de acceso para atributos y funciones son público (*public)*: que establece la visibilidad del atributo o función desde cualquier clase; y privado (*private*): que restringe la visibilidad del atributo o función a únicamente la propia clase, excluyendo a las clases que hereden de la misma. Los niveles de acceso *public, protected* y *private* permiten definir el nivel de encapsulamiento de los atributos y funciones de clases en Java. Los atributos declarados en las líneas 12 y 17 son de tipo *double*, este tipo primitivo permite valores decimales muy grandes. Hay otros tipos de datos primitivos en java, los cuales se muestran en la Tabla 1.

La línea 22 muestra la declaración del constructor de la clase Rectangulo. Un constructor es una función que se define en una clase para inicializar los atributos de los objetos que se crean por medio de la instanciación de la clase. En este caso, el constructor inicializa los atributos base y altura con los valores 2 y 1 respectivamente. Estos valores fijos asignados a los atributos de un objeto durante su creación se denominan valores *por defecto*. En las inicializaciones de los atributos en las líneas 23 y 24 puede verse el uso de la palabra reservada *this,* la cual es una referencia a la instancia actual del objeto.

		NOMBRE	TIPO	OCUPA	RANGO APROXIMADO
TIPOS DE DATOS EN JAVA	**TIPOS PRIMITIVOS** (sin métodos; no son objetos; no necesitan ser instanciados mediante un constructor)	byte	Entero	1 byte	-128 a 127
		short	Entero	2 bytes	-32768 a 32767
		int	Entero	4 bytes	$2*10^9$
		long	Entero	8 bytes	Muy grande
		float	Decimal simple	4 bytes	Muy grande
		double	Decimal doble	8 bytes	Muy grande
		char	Carácter simple	2 bytes	---
		boolean	Valor true o false	1 byte	---
	TIPOS OBJETO (con métodos, necesitan una invocación a un constructor para ser creados)	Tipos de la biblioteca estándar de Java	String (cadenas de texto) Muchos otros (p.ej. Scanner, TreeSet, ArrayList…)		
	TIPOS OBJETO (con métodos, necesitan una invocación a un constructor)	Tipos definidos por el programador / usuario	Cualquiera que se nos ocurra, por ejemplo Taxi, Autobus, Tranvia		
		arrays	Serie de elementos o formación tipo vector o matriz. Lo consideraremos un objeto especial que carece de métodos.		

		Tipos envoltorio o wrapper (Equivalentes a los tipos primitivos pero como objetos.)	Byte
		Tipos envoltorio o wrapper (Equivalentes a los tipos primitivos pero como objetos.)	Short
			Integer
			Long
			Float
			Double
			Character
			Boolean

Tabla 2 Tipos de datos en java (https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=419:tipos-de-datos-java-tipos-primitivos-int-boolean-y-objeto-string-array-o-arreglo-variables-cu00621b&catid=68&Itemid=188)

2. Definición de Comportamiento

A continuación, se describe cómo es posible especificar el comportamiento común a los objetos que se instancian a partir de clases en Java. El ejemplo de la *Tabla 3* es una variante del ejemplo de la Tabla 1 en la que se añaden funciones miembro para el cálculo del perímetro y área del rectángulo; y para la obtención y establecimiento de valores para las propiedades de base y altura.

En la sección anterior se describió como los atributos *base* y *altura* de la clase Rectángulo fueron encapsulados mediante su declaración como atributos protegidos. En esta sección se muestran las maneras en las que estos atributos pueden ser gestionados mediante funciones miembro de la clase. En la línea 33 de la *Tabla 3*, se puede ver una variante del constructor de la clase Rectangulo, para la cual se especifican dos parámetros de entrada: base y altura. Esto permite inicializar los valores de las propiedades del objeto según sea necesario en el momento de su instanciación. Notar que el constructor sin parámetros que se describió en la sección anterior también se encuentra presente en la definición de la clase. La definición de distintas funciones con el mismo nombre y diferentes parámetros de entrada o salida es una técnica que se conoce como sobrecarga de funciones (function overloading). En las líneas 34 y 35, también es posible ver cómo el operador *this* es empleado en este caso para distinguir las referencias a las propiedades base y altura del objeto de los parámetros base y altura aceptados por el constructor.

En las líneas 42 y 51, es posible ver la declaración de las funciones *getBase* y *getAltura,* las cuales permiten obtener el valor de los atributos encapsulados *base* y *altura*, respectivamente. Notar que el nombre de estas funciones está precedido por las palabras *public* y *double,* las cuales indican que las funciones pueden ser accedidas por otras clases, y que estas funciones devuelven un valor de tipo *double*. Además, ver como los comentarios de las líneas 40 y 49 emplean la palabra reservada *@return* para documentar el tipo de datos devuelto por estas funciones. Las líneas 43 y 52 definen el retorno de

los valores de los atributos base y altura, lo cual es definido mediante el uso de la palabra reservada *return.*

Las líneas 59 y 67 definen la declaración de las funciones *setBase* y *setAltura*, los cuales permiten establecer el valor de los atributos base y altura. El nombre de estas funciones se encuentra precedido de las palabras *public* y *void*. Como ya se indicó, *public* especifica la visibilidad de estas funciones desde otras clases. Por su parte *void* (vacío) es la palabra reservada que se emplea para indicar que la función no devuelve ningún valor como resultado de su ejecución.

Las funciones para obtener y establecer el valor de atributos encapsulados por los objetos de una clase se conocen como *getters* y *setters*. Además de ser una práctica adecuada respecto de los preceptos de la POO, estas funciones presentan otras ventajas, tales como la posibilidad de agregar lógica para limitar los valores que pueden ser asignados a los atributos, o bien, permitir la obtención de propiedades derivadas. Una propiedad derivada (ó calculada) no consiste en un atributo almacenado por el objeto, sino uno que puede obtenerse mediante una o más operaciones sobre los atributos almacenados en el mismo. Las líneas 75 y 83 definen la declaración de las funciones públicas *getPerimetro* y *getArea,* que permiten la obtención del perímetro y el área del rectángulo, los cuales son derivados a partir de la base y la altura del rectángulo.

En este ejemplo se introdujeron distintas consideraciones acerca del modo en el que es posible declarar funciones miembro de clases Java. Al conjunto de todas las funciones públicas de una clase se le conoce como la *interfaz* de una clase. Este es un concepto que se elaborará con más profundidad más adelante en esta materia.

```
1.    package poo.figuras.geometricas;
2.
3.    /**
4.    Ejemplo de clase concreta Rectángulo
5.
6.    @author TSTI POO
7.    */
8.    public class Rectangulo {
9.
10.     /*
11.     Atributo protegido (encapsulado) base
12.     */
13.     protected double base;
14.
15.     /**
16.     Atributo protegido (encapsulado) altura
17.     */
18.     protected double altura;
19.
20.     /**
21.     Constructor de la clase Rectangulo
22.     */
23.     public Rectangulo(){
```

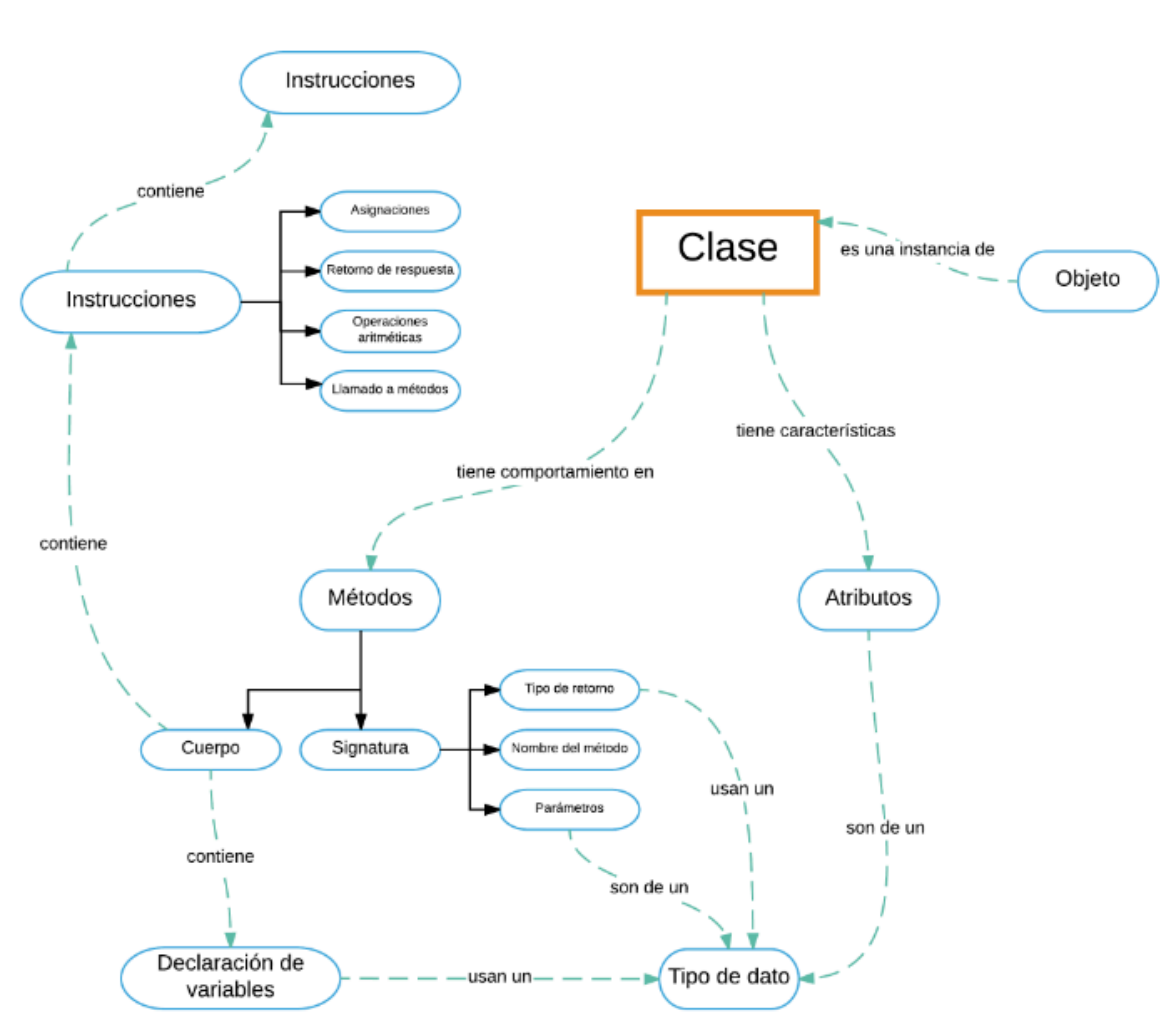
```
24.         this.base = 2;
25.         this.altura = 1;
26.       }
27.
28.       /**
29.       Constructor con inicialización de atributos
30.       @param base
31.       @param altura
32.       */
33.       public Rectangulo(double base, double altura){
34.         this.base = base;
35.         this.altura = altura;
36.       }
37.
38.       /**
39.       Función que permite obtener el valor del atributo encapsulado base
40.       @return double
41.       */
42.       public double getBase() {
43.         return this.base;
44.       }
45.
46.       /**
47.       Función que permite obtener el valor del atributo encapsulado altura
48.       @return double
49.       */
50.       public double getAltura() {
51.         return this.altura;
52.       }
53.
54.       /**
55.       Función que permite establecer el valor del atributo encapsulado
56.       base
57.       */
58.       public void setBase(double base) {
59.         this.base = base;
60.       }
61.
62.       /**
63.       Función que permite establecer el valor del atributo encapsulado
64.       altura
65.       */
66.       public void setAltura(double altura) {
67.         this.altura = altura;
68.       }
69.
70.       /**
71.       Función calcula el perímetro del rectangulo
72.       @return double
```

```
73.      */
74.      public double getPerimetro(){
75.        return this.base * 2 + this.altura * 2;
76.      }
77.
78.      /**
79.      Función que calcula el área del rectángulo
80.      @return double
81.      */
82.      public double getArea(){
83.        return this.base * this.altura;
84.      }
85.   }
86.
```

Tabla 3. Declaración de clase Java con Funciones Miembro

RESUMEN de la Unidad 1

Hemos introducido conceptos básicos de la programación orientada a objetos como un primer acercamiento a la materia. A continuación tenemos un mapa mental que resume los conceptos tratados:



Como Actividad Propuesta, realizar la guía práctica 1 publicada en el Campus de la materia.

Referencias

1. Joyanes Aguilar, Luis. *Programación orientada a objetos* (No. 004.652. 5). McGraw-Hill Interamericana.

2. Jorge A. Villalobos. Rubby Casalla, *Fundamentos de Programación*. Departamento de ingeniería de sistemas y computación..Universidad de los Andes.